

Part A

CDAC MUMBAI - Concepts of Operating System Assignment 2 - Solutions

1. `echo "Hello, World!"`
 - Prints "Hello, World!" to the terminal.
2. `name="Productive"`
 - Assigns the value "Productive" to the variable ``name``.
3. `touch file.txt`
 - Creates an empty file named ``file.txt`` if it does not exist or updates its timestamp if it does.
4. `ls -a`
 - Lists all files and directories, including hidden ones (those starting with ``.``).
5. `rm file.txt`
 - Deletes the file ``file.txt``.
6. `cp file1.txt file2.txt`
 - Copies ``file1.txt`` to ``file2.txt``.
7. `mv file.txt /path/to/directory/`
 - Moves ``file.txt`` to the specified directory.
8. `chmod 755 script.sh`
 - Gives the owner full permissions (read, write, execute) and grants read & execute permissions to the group and others.
9. `grep "pattern" file.txt`
 - Searches for "pattern" in ``file.txt`` and displays matching lines.
10. `kill PID`
 - Terminates the process with the specified ``PID``.
11. `mkdir mydir && cd mydir && touch file.txt && echo "Hello, World!" > file.txt && cat file.txt`
 - Creates a directory ``mydir``, moves into it, creates ``file.txt``, writes "Hello, World!" into it, and then displays its contents.
12. `ls -l | grep ".txt"`
 - Lists all files in long format and filters those containing ".txt" in their names.
13. `cat file1.txt file2.txt | sort | uniq`
 - Concatenates ``file1.txt`` and ``file2.txt``, sorts the combined content, and removes duplicate lines.

14. `ls -l | grep "^d"`
 - Lists only directories by filtering lines that start with `d` in the detailed `ls -l` output.
15. `grep -r "pattern" /path/to/directory/`
 - Recursively searches for "pattern" in all files within the specified directory.
16. `cat file1.txt file2.txt | sort | uniq -d`
 - Concatenates `file1.txt` and `file2.txt`, sorts the output, and displays only duplicate lines.
17. `chmod 644 file.txt`
 - Sets read & write permissions for the owner and read-only permissions for group and others.
18. `cp -r source_directory destination_directory`
 - Recursively copies `source\_directory` and all its contents to `destination\_directory`.
19. `find /path/to/search -name "*.txt"`
 - Searches for all `.txt` files in the specified directory and its subdirectories.
20. `chmod u+x file.txt`
 - Grants execute permission to the owner of `file.txt`.
21. `echo $PATH`
 - Displays the system's `PATH` environment variable, which lists directories where executable files are located.

Part B

22. Identify True or False:
 - Is is used to list files and directories in a directory. • Answer: True
23. mv is used to move files and directories.
 - Answer: True
24. cd is used to copy files and directories.
 - Answer: False
25. pwd stands for 'print working directory' and displays the current directory.
 - Answer: True
26. grep is used to search for patterns in files.
 - Answer: True

27. `chmod 755 file.txt` gives read, write, and execute permissions to the owner, and read and execute permissions to group and others.
• Answer: True
28. `mkdir -p directory1/directory2` creates nested directories, creating `directory2` inside `directory1` if `directory1` does not exist.
• Answer: True
29. `rm -rf file.txt` deletes a file forcefully without confirmation. • Answer: True
- Identify the Incorrect Commands:
30. `chmodx` is used to change file permissions.
• Answer: Incorrect
31. • `cpy` is used to copy files and directories.
• Answer: Incorrect
32. `mkfile` is used to create a new file.
• Answer: Incorrect
33. `catx` is used to concatenate files.
• Answer: Incorrect
34. `rn` is used to rename files.
• Answer: Incorrect

Part C

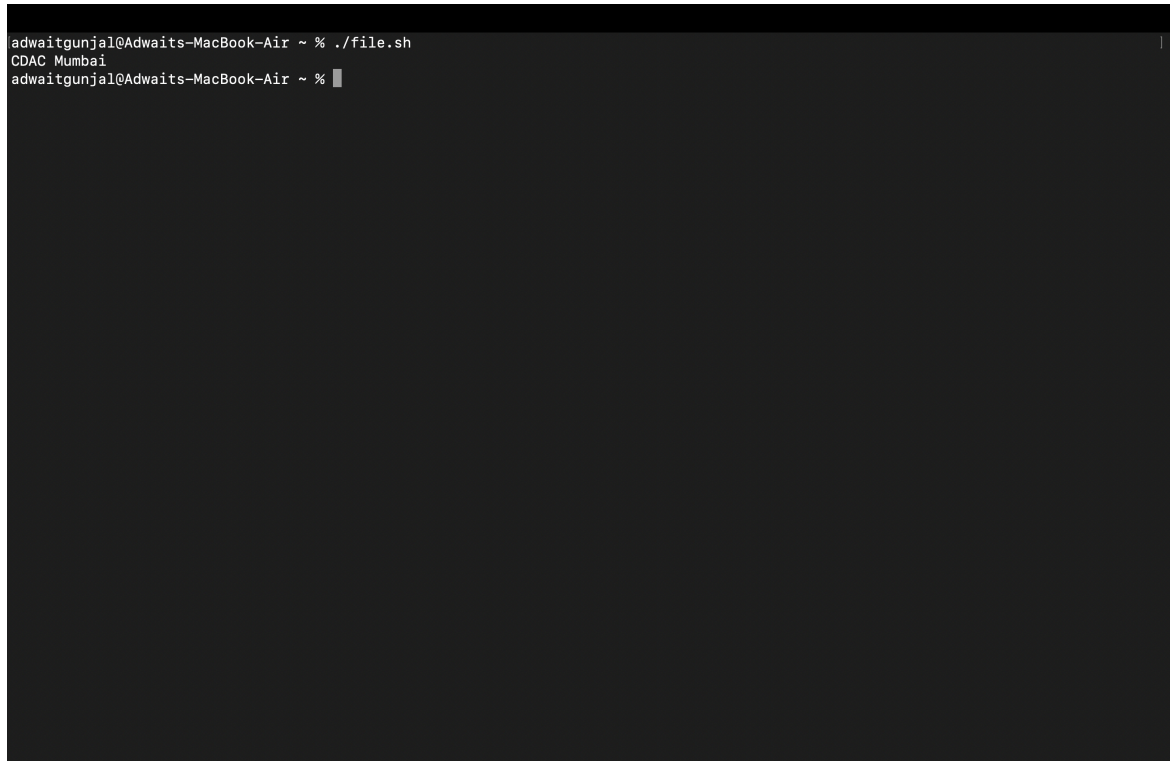
1. Write a shell script that prints 'Hello, World!' to the terminal.
`echo Hello, World!`

```
adwaitgunjal@Adwails-MacBook-Air ~ % ./file.sh
Hello, World!
adwaitgunjal@Adwails-MacBook-Air ~ %
```

2. Declare a variable named 'name' and assign the value 'CDAC Mumbai' to it. Print the value of the variable.

```
name="CDAC Mumbai"
```

```
echo $name
```

A screenshot of a terminal window with a dark background. The prompt is 'adwaitgunjal@Adwails-MacBook-Air ~ %'. The user enters './file.sh', and the output 'CDAC Mumbai' is displayed on the next line. The prompt returns to 'adwaitgunjal@Adwails-MacBook-Air ~ %' with a cursor.

```
adwaitgunjal@Adwails-MacBook-Air ~ % ./file.sh
CDAC Mumbai
adwaitgunjal@Adwails-MacBook-Air ~ %
```

3. Write a shell script that takes a number as input from the user and prints it.
read -p “

Enter a number: " num

echo "You entered: \$num”

```
adwaitgunjal@Adwait's-MacBook-Air ~ % ./file.sh
Enter a number: 10
You entered: 10
adwaitgunjal@Adwait's-MacBook-Air ~ %
```

4. Write a shell script that performs addition of two numbers (e.g., 5 and 3) and prints the result.

```
a=5
```

```
b=3
```

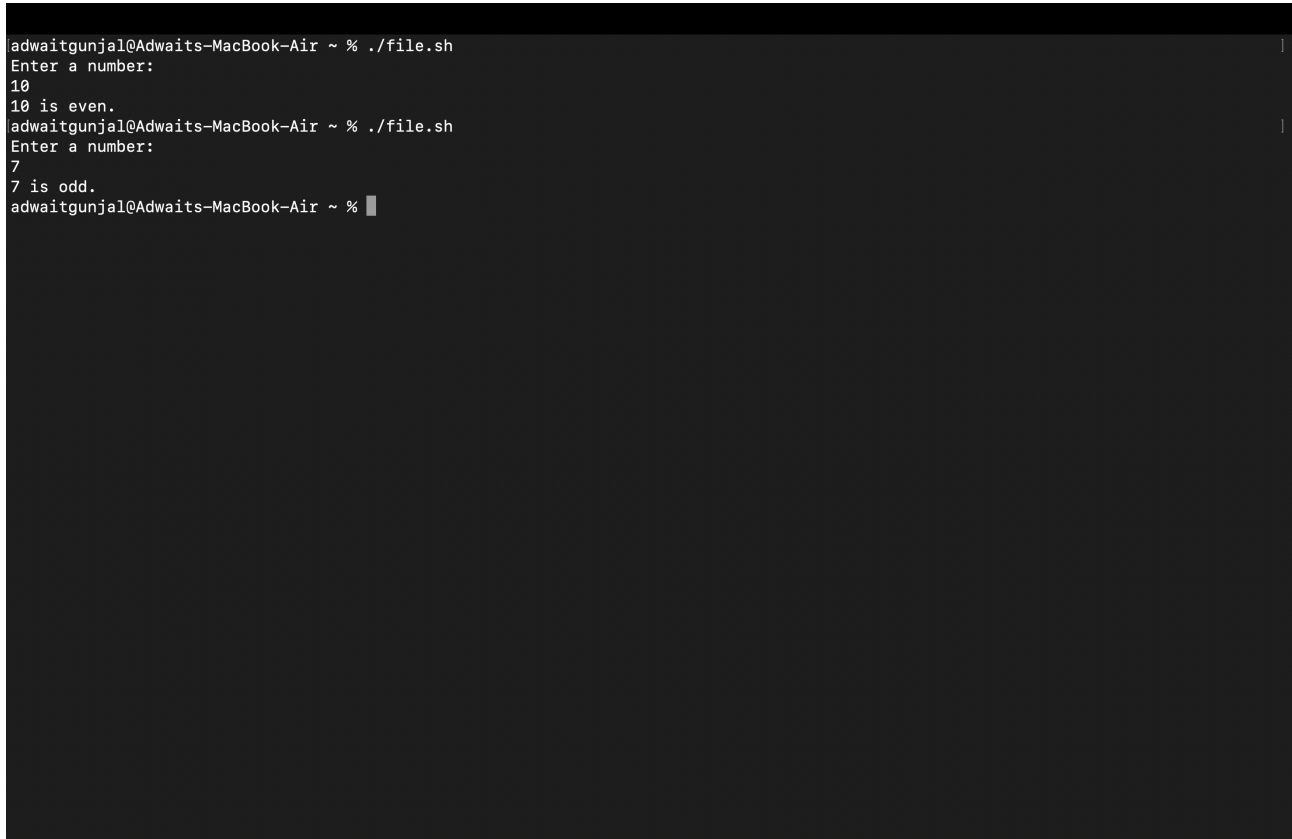
```
sum=$((a + b))
```

```
echo Sum: $sum
```

```
adwaitgunjal@Adwails-MacBook-Air ~ % ./file.sh
Sum: 8
adwaitgunjal@Adwails-MacBook-Air ~ %
```

5. Write a shell script that takes a number as input and prints 'Even' if it is even, otherwise prints 'Odd'.

```
echo "Enter a number: "  
  
read num  
  
if (( num % 2 == 0 )); then  
    echo "$num is even."  
  
else  
    echo "$num is odd."  
  
fi
```



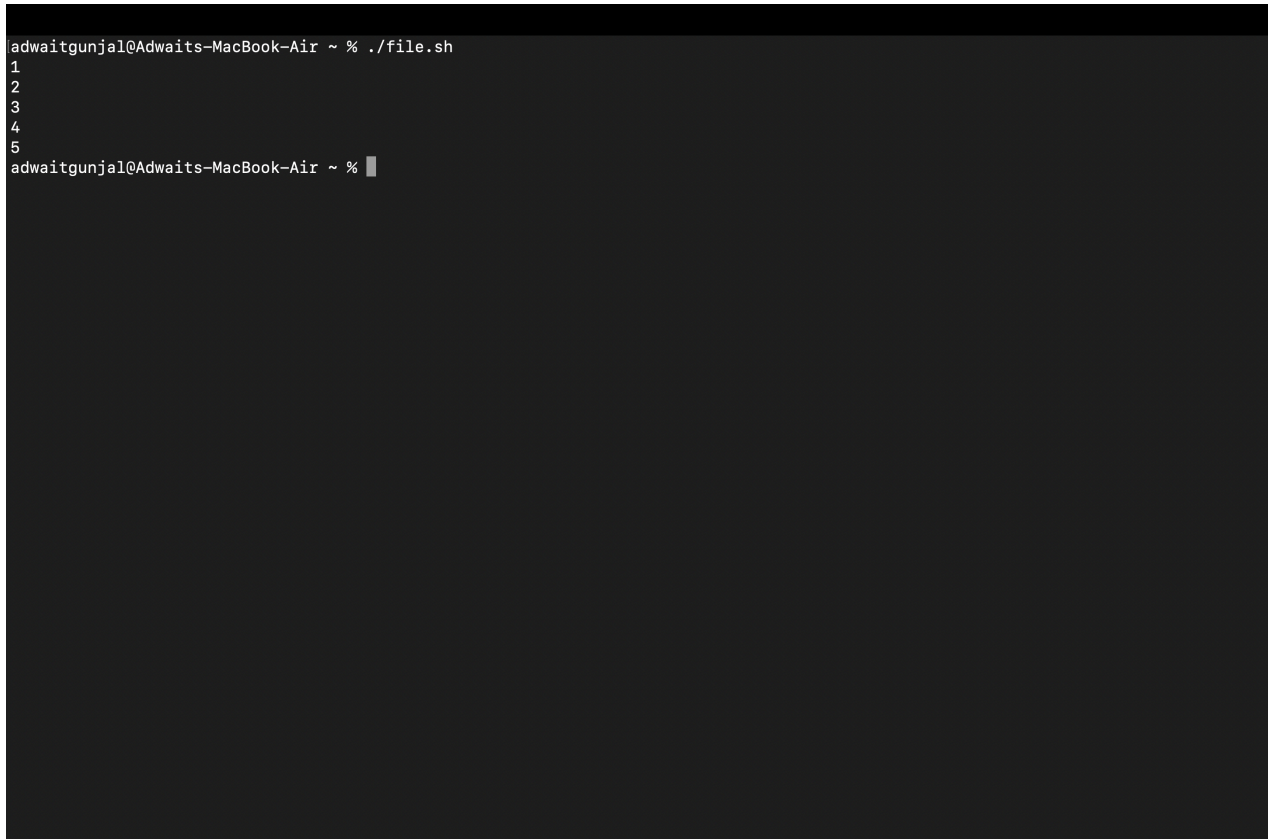
```
adwaitgunjal@Adwails-MacBook-Air ~ % ./file.sh  
Enter a number:  
10  
10 is even.  
adwaitgunjal@Adwails-MacBook-Air ~ % ./file.sh  
Enter a number:  
7  
7 is odd.  
adwaitgunjal@Adwails-MacBook-Air ~ %
```

6. Write a shell script that uses a for loop to print numbers from 1 to 5.

```
for i in {1..5}; do
```

```
echo $i
```

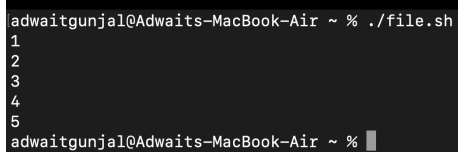
```
done
```

A terminal window with a dark background. The prompt is 'adwaitgunjal@Adwait-MacBook-Air ~ %'. The user has entered './file.sh'. The output shows the numbers 1, 2, 3, 4, and 5, each on a new line. The prompt is now 'adwaitgunjal@Adwait-MacBook-Air ~ %' with a cursor.

```
adwaitgunjal@Adwait-MacBook-Air ~ % ./file.sh
1
2
3
4
5
adwaitgunjal@Adwait-MacBook-Air ~ %
```


7. Shell script that uses a while loop to print numbers from 1 to 5.

```
num=1
while [ $num -le 5 ]
do
    echo $num
    ((num++))
done
```

A terminal window with a dark background. The prompt is 'adwaitgunjal@Adwait's-MacBook-Air ~ %'. The user has entered './file.sh'. The output shows the numbers 1, 2, 3, 4, and 5, each on a new line. The prompt is now 'adwaitgunjal@Adwait's-MacBook-Air ~ %' with a cursor.

```
adwaitgunjal@Adwait's-MacBook-Air ~ % ./file.sh
1
2
3
4
5
adwaitgunjal@Adwait's-MacBook-Air ~ %
```

8. Write a shell script that checks if a file named 'file.txt' exists in the current directory.

```
FILE="file.txt"
```

```
if [ -e "$FILE" ]; then
```

```
    echo "The file exists."
```

```
else
```

```
    echo "The file does not exist."
```

```
fi
```

```
adwaitgunjal@Adwait's-MacBook-Air ~ % ls
Applications      Downloads         Music             eclipse           iCloud Drive (Archive)
Desktop           Library          Pictures          eclipse-workspace upsc
Documents         Movies           Public            file.sh
adwaitgunjal@Adwait's-MacBook-Air ~ % ./file.sh
The file does not exist.
adwaitgunjal@Adwait's-MacBook-Air ~ % touch file.txt
adwaitgunjal@Adwait's-MacBook-Air ~ % ls
Applications      Downloads         Music             eclipse           file.txt
Desktop           Library          Pictures          eclipse-workspace iCloud Drive (Archive)
Documents         Movies           Public            file.sh           upsc
adwaitgunjal@Adwait's-MacBook-Air ~ % ./file.sh
The file exists.
adwaitgunjal@Adwait's-MacBook-Air ~ %
```

Part E

1. Consider the following processes with arrival times and burst times: | Process | Arrival Time | Burst Time | |-----|-----|-----|

| P1 | 0 | 5 |

| P2 | 1 | 3 |

| P3 | 2 | 6 |

Calculate the average waiting time using First-Come, First-Served (FCFS) scheduling.

Final Answer: 3.33 units

2. Consider the following processes with arrival times and burst times:

| Process | Arrival Time | Burst Time | |-----|-----|-----|

| P1 | 0 | 3 |

| P2 | 1 | 5 |

| P3 | 2 | 1 |

| P4 | 3 | 4 |

Calculate the average turnaround time using Shortest Job First (SJF) scheduling.

Final Answer: 5.5 units

3. Consider the following processes with arrival times, burst times, and priorities (lower number indicates higher

priority):

| Process | Arrival Time | Burst Time | Priority |

|-----|-----|-----|-----|

| P1 | 0 | 6 | 3 |

| P2 | 1 | 4 | 1 |

| P3 | 2 | 7 | 4 |

| P4 | 3 | 2 | 2 |

Calculate the average waiting time using Priority Scheduling.

• Final Answer: 5 units

4. Consider the following processes with arrival times and burst times, and the time quantum for Round Robin scheduling is 2 units:

| Process | Arrival Time | Burst Time |

|-----|-----|-----|

| P1 | 0 | 4 |

| P2 | 1 | 5 |

| P3 | 2 | 2 |

| P4 | 3 | 3 |

Calculate the average turnaround time using Round Robin scheduling.

- Final Answer: 6.25 units

5. Consider a program that uses the fork() system call to create a child process. Initially, the parent process has a variable x with a value of 5. After forking, both the parent and child processes increment the value of x by 1. What will be the final values of x in the parent and child processes after the fork() call?

- Final Answer: Parent: 6, Child: 6